

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Conception et Développement d'un CRM Intelligent

**Intégrant un Module de Prédiction des
Ventes**

**Basé sur le Machine Learning
pour le E-Commerce**

Projet de Stage

Spécialité : Data Science

Entreprise d'accueil : BAT PROJET ENGINEERING

Représentée par Mr. ATALLAH Mohammed

Réalisé par :

Rezaiguia Soltane Tadj Eddine
(DS)

Bellatreche Mohamed Amine
(DS)

Khelifi Ayyoub (CS)

Brahim Soheib (AI)

Encadré par :

Dr. Khellat Souad

Année Universitaire 2025–2026

Table des matières

Introduction Générale	1
1 Contexte et Problématique	2
1.1 Le e-commerce en Algérie	2
1.2 Problématique du COD	2
1.3 Objectifs du projet	3
2 État de l’Art	4
2.1 CRM intelligent et aide à la décision	4
2.2 Machine Learning pour la prédiction des ventes	4
2.2.1 Gradient Boosting : CatBoost, LightGBM, XGBoost	4
2.2.2 Séries temporelles : LightGBM, Prophet et Amazon Chronos . .	5
2.2.3 Clustering : Approche hybride HDBSCAN / KMeans	6
2.2.4 Analyse RFM	6
2.2.5 Rééchantillonnage : SMOTE et ADASYN	6
2.3 Métriques d’évaluation	7
2.3.1 Classification (prédiction de risque)	7
2.3.2 Régression / Séries temporelles (prévision de demande)	7
3 Conception et Architecture	8
3.1 Architecture globale du système	8
3.2 Conception de la base de données	9
3.2.1 Schéma relationnel	9
3.2.2 Tables principales	10
3.2.3 Cycle de vie d’une commande COD	10
3.3 Architecture du module ML	11
3.3.1 Intégration IA dans le système	11
3.3.2 Structure du microservice	12
3.3.3 Endpoints de l’API ML	13
3.3.4 Intégration avec le backend PHP	13
4 Réalisation	15
4.1 Technologies utilisées	15

4.2	Le CRM : Modules fonctionnels	15
4.3	Jeu de données : Olist Brazilian E-Commerce	16
4.3.1	Transformation et mapping	16
4.4	Réentraînement des modèles	17
5	Modèles de Machine Learning, Évaluation et Résultats	19
5.1	Pipeline Machine Learning	19
5.2	Prédiction du risque de livraison	20
5.2.1	Formulation du problème	20
5.2.2	Ingénierie des features	20
5.2.3	Modèles entraînés et comparaison	21
5.2.4	Scoring des commandes existantes	24
5.2.5	Catégories de risque et aide à la décision	24
5.2.6	Actions automatisées du workflow	24
5.2.7	Pourquoi la statistique J de Youden ?	25
5.3	Segmentation client — Approche hybride	25
5.3.1	Méthodologie	25
5.3.2	Résultats de la segmentation	26
5.3.3	Interprétation pour l'aide à la décision	27
5.4	Prévision de la demande : Benchmark et LightGBM	27
5.4.1	Préparation des données temporelles	27
5.4.2	Benchmark comparatif de 5 modèles	27
5.4.3	Choix du modèle : LightGBM	28
5.4.4	Ingénierie des features temporelles	28
5.4.5	Modèles par catégorie	29
5.4.6	Date de départ flexible pour la prévision	30
5.4.7	Pourquoi les modèles fondamentaux (HuggingFace) ne sont pas adaptés	30
5.5	Intégration LLM : Google Gemini	31
5.6	Synthèse comparative des modèles	32
5.7	Sécurité et protection des données	32
5.7.1	Volet cybersécurité (contribution de l'équipe)	33
5.7.2	Méthodologie d'audit	33
5.7.3	Architecture de sécurisation implémentée	33
5.7.4	Synthèse des vulnérabilités corrigées	34
5.7.5	Vérification du code source (17/04/2026)	35
5.7.6	Bonnes pratiques de déploiement	35
	Conclusion Générale	37

Bibliographie	39
A Annexe : Exemple de requête et réponse de l'API	41
A.1 Prédiction du risque d'une commande	41
A.2 Prévision de la demande	42

Table des figures

3.1	Architecture globale du système	8
3.2	Schéma relationnel de la base de données	9
3.3	Cycle de vie d'une commande COD avec intégration IA	11
3.4	Flux d'intégration IA dans le CRM	12
5.1	Pipeline Machine Learning : de la donnée brute au déploiement	19

Liste des tableaux

3.1	Description des tables de la base de données	10
3.2	Endpoints REST du module ML	13
4.1	Stack technologique du projet	15
4.2	Mapping des données Olist vers le schéma CRM	16
4.3	Endpoints de réentraînement et gestion des modèles	18
5.1	Features utilisées pour la prédiction du risque (31 features, sans fuite de données)	20
5.2	Répartition des données d'entraînement et de test (cible propre)	22
5.3	Hyperparamètres optimisés par Optuna (LightGBM)	22
5.4	Comparaison des performances des modèles de classification	22
5.5	Matrice de confusion du modèle ensemble sur l'ensemble de test (19 543 échantillons)	23
5.6	Impact cumulatif des optimisations sur l'AUC-ROC	23
5.7	Détection des échecs de livraison : avant et après optimisation	23
5.8	Catégories de risque et recommandations	24
5.9	Actions de workflow selon le score de succès	25
5.10	Sélection automatique de l'algorithme selon la taille du dataset	26
5.11	Segments clients identifiés par l'approche hybride	26
5.12	Benchmark des modèles de prévision (horizon 30 jours, trié par MAE)	27
5.13	Features utilisées pour la prévision de la demande (LightGBM)	29
5.14	Modèles de prévision par catégorie de produit	30
5.15	Synthèse des modèles développés	32
5.16	Mécanismes d'authentification et d'autorisation par couche	33
5.17	Protections techniques contre les attaques applicatives	34
5.18	Synthèse de sévérité des corrections appliquées	34
5.19	Points de correction intégrés dans le projet	35

Introduction Générale

Le commerce électronique connaît une croissance exponentielle en Algérie, particulièrement dans le modèle *Cash-on-Delivery* (COD), où le paiement s'effectue à la livraison. Ce modèle, bien qu'adapté au contexte local où les solutions de paiement électronique restent limitées, engendre des défis majeurs : un taux d'échec de livraison de 30 à 50%, des retours coûteux qui érodent les marges, et une gestion inefficace des commandes.

Face à ces problématiques, les systèmes de gestion de la relation client (CRM) traditionnels s'avèrent insuffisants. Ils se limitent à la gestion passive des données sans exploiter le potentiel prédictif qu'elles renferment. L'intégration de l'intelligence artificielle, et plus spécifiquement du Machine Learning, dans ces systèmes ouvre de nouvelles perspectives pour transformer les données historiques en informations décisionnelles actionnables.

Le présent travail s'inscrit dans cette démarche et vise à concevoir et développer un CRM intelligent pour le e-commerce COD, intégrant un module de prédiction basé sur le Machine Learning. Ce module comprend trois axes principaux :

1. **La prédiction du risque de livraison** : un ensemble de modèles (CatBoost, LightGBM, XGBoost) pour évaluer la probabilité de succès de chaque commande.
2. **La segmentation client** : une approche hybride HDBSCAN (grands jeux de données) / KMeans (petits jeux de données) pour identifier automatiquement des groupes de clients aux comportements similaires.
3. **La prévision de la demande** : un modèle LightGBM avec covariables du calendrier islamique pour anticiper les tendances de vente futures.

Ce rapport est structuré en cinq chapitres. Le premier présente le contexte et la problématique. Le deuxième dresse un état de l'art des techniques utilisées. Le troisième détaille la conception et l'architecture du système. Le quatrième décrit la réalisation technique. Le cinquième présente les modèles de Machine Learning, leur évaluation comparative et l'interprétation des résultats.

Chapitre 1

Contexte et Problématique

1.1 Le e-commerce en Algérie

L'Algérie possède un marché e-commerce en pleine expansion, porté par une population jeune et connectée. Cependant, plusieurs spécificités distinguent ce marché :

- **Le modèle COD dominant** : En l'absence de solutions de paiement électronique largement adoptées, plus de 90% des transactions se font en Cash-on-Delivery.
- **La géographie logistique** : Le territoire est divisé en 58 wilayas, regroupées en 3 zones d'expédition avec des coûts et délais variables.
- **La monnaie locale** : Toutes les transactions s'effectuent en Dinar Algérien (DZD).

1.2 Problématique du COD

Le modèle COD présente des défis spécifiques :

1. **Taux d'échec élevé** : 30 à 50% des commandes COD ne sont jamais livrées avec succès (refus, absence, adresse incorrecte).
2. **Coûts de retour** : Chaque commande échouée engendre des frais de transport aller-retour sans recette.
3. **Gestion manuelle** : Les confirmations téléphoniques et le suivi sont souvent manuels et chronophages.
4. **Décisions non informées** : Les gestionnaires manquent d'outils analytiques pour prioriser les commandes ou anticiper les tendances.

1.3 Objectifs du projet

Les objectifs de ce projet sont :

1. **Concevoir un CRM fonctionnel** : Gestion complète des clients, commandes, produits, stocks, livraisons et retours, avec une architecture multi-tenant.
2. **Concevoir rigoureusement la base de données** : Schéma normalisé, audit trails, intégrité référentielle, support multilingue.
3. **Mettre en place un tableau de bord analytique** : Visualisation des KPIs, tendances, et métriques en temps réel.
4. **Développer et comparer des modèles de ML** : Au moins deux modèles pour la prédiction, avec évaluation rigoureuse.
5. **Interpréter les résultats** : Fournir des recommandations actionnables pour l'aide à la décision.

Chapitre 2

État de l'Art

2.1 CRM intelligent et aide à la décision

Un CRM (Customer Relationship Management) est un système de gestion de la relation client permettant de centraliser, organiser et exploiter les données clients. Un CRM *intelligent* se distingue par l'intégration de capacités analytiques et prédictives, transformant les données passives en informations actionnables.

L'évolution des CRM suit trois générations :

1. **CRM opérationnel** : Gestion basique des contacts et transactions.
2. **CRM analytique** : Tableaux de bord et rapports statistiques.
3. **CRM intelligent** : Intégration du ML pour la prédiction et la recommandation.

2.2 Machine Learning pour la prédiction des ventes

La prédiction des ventes est un problème classique du ML qui peut être abordé sous plusieurs angles :

2.2.1 Gradient Boosting : CatBoost, LightGBM, XGBoost

Les algorithmes de gradient boosting sont considérés comme l'état de l'art pour les données tabulaires [8]. Ils construisent un ensemble d'arbres de décision de manière séquentielle, chaque arbre corrigeant les erreurs du précédent.

- **XGBoost** [3] : Implémentation optimisée de gradient boosting avec régularisation L1/L2. Utilise une approximation de second ordre (Newton) pour l'optimisation.

- **LightGBM** [4] : Utilise le *Gradient-based One-Side Sampling* (GOSS) et l'*Exclusive Feature Bundling* (EFB) pour accélérer l'entraînement. Croissance par feuille (*leaf-wise*) plutôt que par niveau.
- **CatBoost** [5] : Gestion native des variables catégorielles via *ordered target encoding*. Utilise des permutations aléatoires pour réduire le biais de prédiction.

L'approche *ensemble* combine les prédictions de ces trois modèles par vote pondéré (*soft voting*), ce qui améliore la robustesse et réduit la variance.

2.2.2 Séries temporelles : LightGBM, Prophet et Amazon Chronos

Plusieurs approches ont été évaluées lors du benchmark de modèles de prévision :

Prophet [6] est un modèle de prévision de séries temporelles développé par Meta. Il décompose la série en trois composantes additives ou multiplicatives :

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \quad (2.1)$$

où $g(t)$ est la tendance (*growth*), $s(t)$ la saisonnalité, $h(t)$ les effets de jours fériés, et ϵ_t le terme d'erreur.

Amazon Chronos [10] est une famille de modèles de langage pré-entraînés pour la prévision de séries temporelles. Basé sur l'architecture T5, Chronos tokenise les valeurs réelles en bins discrets et offre une prévision *zero-shot* sans entraînement spécifique au domaine.

LightGBM pour les séries temporelles [4] : LightGBM, initialement conçu pour les données tabulaires, peut être adapté à la prévision de séries temporelles grâce à une ingénierie de features temporelles. Le modèle utilise des *lag features* (retards de 1, 7, 14 et 28 jours), des statistiques glissantes (moyenne et écart-type sur des fenêtres de 7, 14 et 28 jours), des features calendaires (jour de la semaine, mois, jours fériés/week-end), ainsi que des **covariables du calendrier algérien** : événements islamiques (Ramadan, Aïd el-Fitr, Aïd el-Adha, Mawlid, Nouvel An islamique) calculés via **hijri-converter**, et jours fériés nationaux (1^{er} janvier, Yennayer, 1^{er} mai, 5 juillet, 1^{er} novembre). La prévision multi-étape est réalisée de manière récursive : chaque prédiction est réinjectée comme feature pour l'horizon suivant. Ce modèle a été sélectionné après un benchmark comparatif de 5 approches, grâce à sa capacité à intégrer des covariables exogènes pertinentes pour le marché algérien.

2.2.3 Clustering : Approche hybride HDBSCAN / KMeans

Notre système utilise une approche hybride adaptative pour la segmentation client :

- **HDBSCAN** pour les grands jeux de données (≥ 1000 clients) : détection automatique du nombre de clusters, identification des points de bruit, gestion des clusters de densités et tailles variables. Un seul hyperparamètre principal : `min_cluster_size`.
- **KMeans** pour les petits jeux de données (< 1000 clients) : sélection automatique du K optimal via le score de silhouette (test de $K = 3$ à 7), plus robuste avec peu de données où HDBSCAN produirait trop de bruit.

Cette approche garantit que la segmentation fonctionne correctement quelle que soit la taille de l'entreprise, des petites boutiques aux grandes plateformes e-commerce.

2.2.4 Analyse RFM

L'analyse RFM (Recency, Frequency, Monetary) est une technique de segmentation client classique :

- **Récence (R)** : Nombre de jours depuis le dernier achat
- **Fréquence (F)** : Nombre total de commandes
- **Montant (M)** : Valeur totale des achats (en DZD)

Ces trois dimensions sont utilisées comme features d'entrée pour l'algorithme de clustering.

2.2.5 Rééchantillonnage : SMOTE et ADASYN

SMOTE (*Synthetic Minority Oversampling Technique*) [1] est une technique d'augmentation des données qui génère des exemples synthétiques pour la classe minoritaire par interpolation linéaire entre des voisins proches :

$$x_{new} = x_i + \lambda \cdot (x_k - x_i), \quad \lambda \in [0, 1] \quad (2.2)$$

où x_i est un exemple de la classe minoritaire et x_k est l'un de ses k plus proches voisins.

ADASYN (*Adaptive Synthetic Sampling*) [2] est une amélioration de SMOTE qui génère davantage d'exemples synthétiques pour les instances difficiles à apprendre (proches de la frontière de décision). La densité de génération est proportionnelle au ratio de voisins de la classe majoritaire :

$$g_i = \frac{r_i}{\sum_j r_j} \times G, \quad r_i = \frac{\Delta_i}{k} \quad (2.3)$$

où Δ_i est le nombre de voisins de la classe majoritaire parmi les k plus proches voisins de x_i , et G est le nombre total d'exemples synthétiques à générer.

Dans notre contexte, le dataset Olist présente un déséquilibre sévère (98,7% livrées, 1,3% échouées après nettoyage de la cible). Après expérimentation comparative entre SMOTE et ADASYN, nous avons retenu **ADASYN** avec `sampling_strategy=0.3`, car il concentre la génération synthétique sur les cas les plus informatifs, améliorant significativement le rappel de la classe minoritaire.

2.3 Métriques d'évaluation

2.3.1 Classification (prédiction de risque)

- **AUC-ROC** : Aire sous la courbe ROC, mesure la capacité discriminante du modèle indépendamment du seuil de décision.
- **Précision** : $\frac{VP}{VP+FP}$ — proportion de prédictions positives correctes.
- **Rappel (Recall)** : $\frac{VP}{VP+FN}$ — proportion de positifs correctement identifiés.
- **F1-Score** : Moyenne harmonique de la précision et du rappel : $F1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$
- **Exactitude (Accuracy)** : $\frac{VP+VN}{Total}$ — taux de bonnes prédictions.

2.3.2 Régression / Séries temporelles (prévision de demande)

- **MAE** (Mean Absolute Error) : $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **RMSE** (Root Mean Squared Error) : $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
- **MAPE** (Mean Absolute Percentage Error) : $\frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$

Chapitre 3

Conception et Architecture

3.1 Architecture globale du système

Le système adopté suit une architecture en microservices, composée de trois composants principaux :

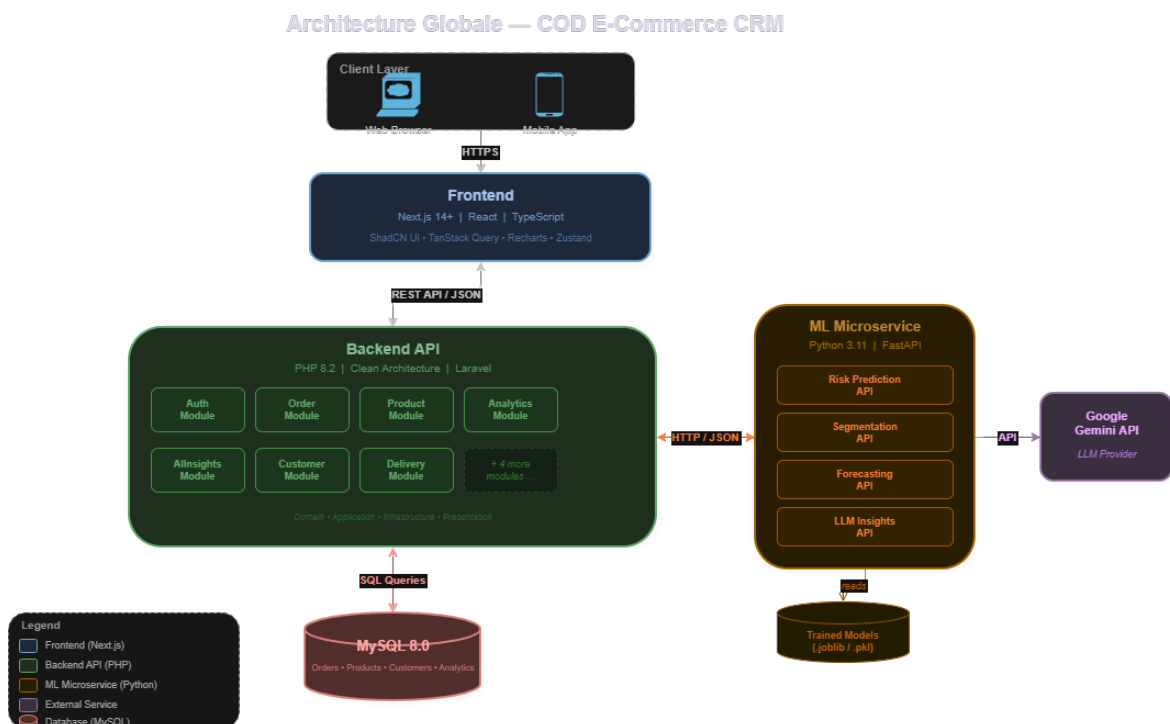


FIGURE 3.1 – Architecture globale du système

Le système repose sur une communication REST/JSON entre les composants. Le backend PHP (Clean Architecture, pattern Controller-Service-Repository) orchestre les échanges avec la base MySQL 8.0 (UTF8MB4 pour le multilingue arabe/français) et délègue les traitements analytiques au microservice ML Python (FastAPI). Ce der-

nière intègre un modèle LightGBM avec covariables du calendrier algérien (islamique + jours fériés nationaux) pour la prévision de la demande, ainsi que l'API Google Gemini (`gemini-2.5-flash`) pour la génération d'insights en langage naturel.

3.2 Conception de la base de données

La base de données MySQL comprend **12 tables** réparties en 5 fichiers de migration, avec une architecture multi-tenant où chaque table opérationnelle est isolée par `store id`.

3.2.1 Schéma relationnel

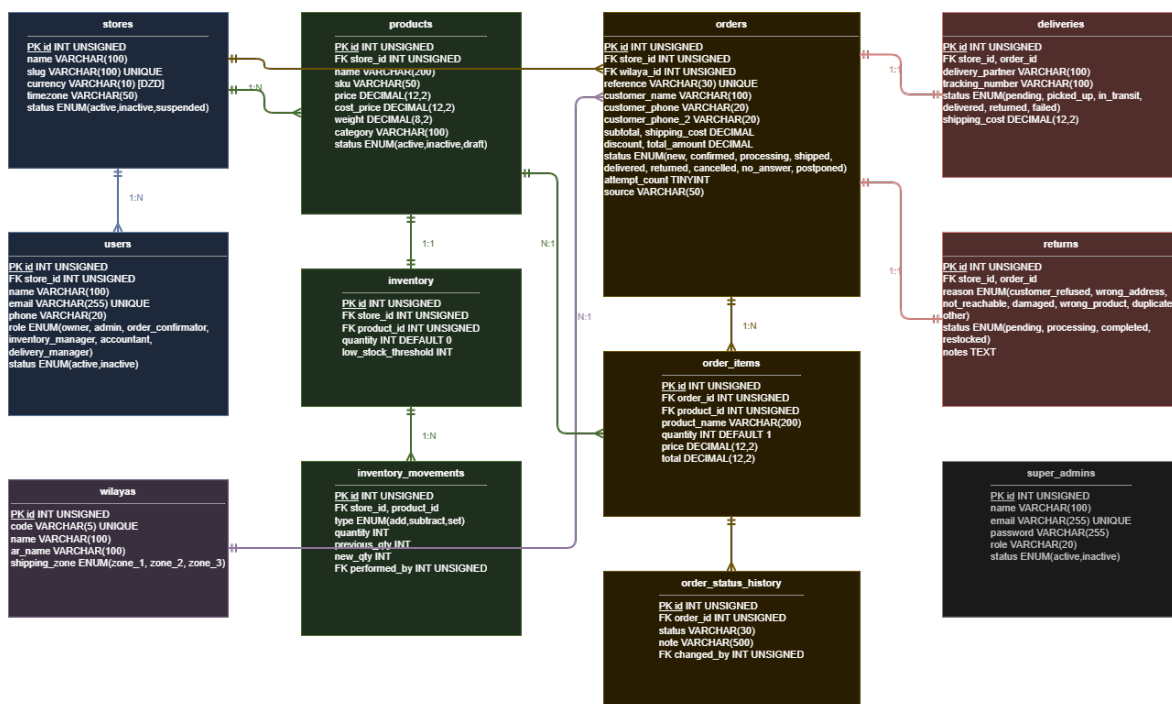


FIGURE 3.2 – Schéma relationnel de la base de données

3.2.2 Tables principales

TABLE 3.1 – Description des tables de la base de données

Table	Clés	Description
<code>stores</code>	PK : id	Magasins (racine multi-tenant)
<code>wilayas</code>	PK : id	58 wilayas d’Algérie, 3 zones d’expédition
<code>users</code>	PK : id, FK : store_id	Utilisateurs avec 6 rôles RBAC
<code>products</code>	PK : id, FK : store_id	Catalogue produits (prix, catégorie, poids)
<code>inventory</code>	PK : id, FK : product_id	Suivi des stocks avec seuil d’alerte
<code>inventory_movements</code>	PK : id	Historique des mouvements de stock
<code>orders</code>	PK : id, FK : store_id, wilaya_id	Commandes avec 9 statuts possibles
<code>order_items</code>	PK : id, FK : order_id	Articles de commande
<code>order_status_history</code>	PK : id, FK : order_id	Piste d’audit des changements de statut
<code>deliveries</code>	PK : id, FK : order_id	Gestion des livraisons (6 statuts)
<code>returns</code>	PK : id, FK : order_id	Gestion des retours (7 raisons)
<code>super_admins</code>	PK : id	Administrateurs de la plateforme

3.2.3 Cycle de vie d’une commande COD

La table `orders` utilise un champ `status` de type ENUM avec 9 états possibles reflétant le cycle de vie complet d’une commande COD :

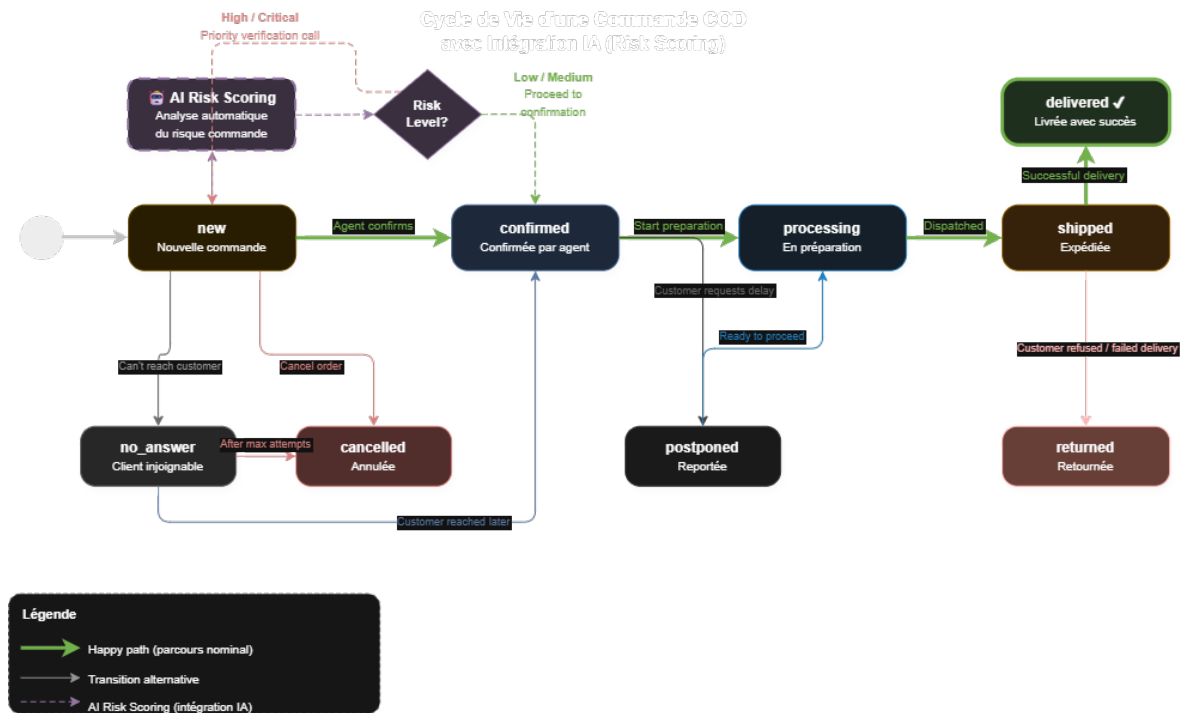


FIGURE 3.3 – Cycle de vie d'une commande COD avec intégration IA

3.3 Architecture du module ML

Le module ML est un microservice Python autonome communiquant avec le backend PHP via API REST.

3.3.1 Intégration IA dans le système

La figure 3.4 présente les trois flux d'analyse IA intégrés dans le CRM, ainsi que le composant LLM pour la génération d'insights.

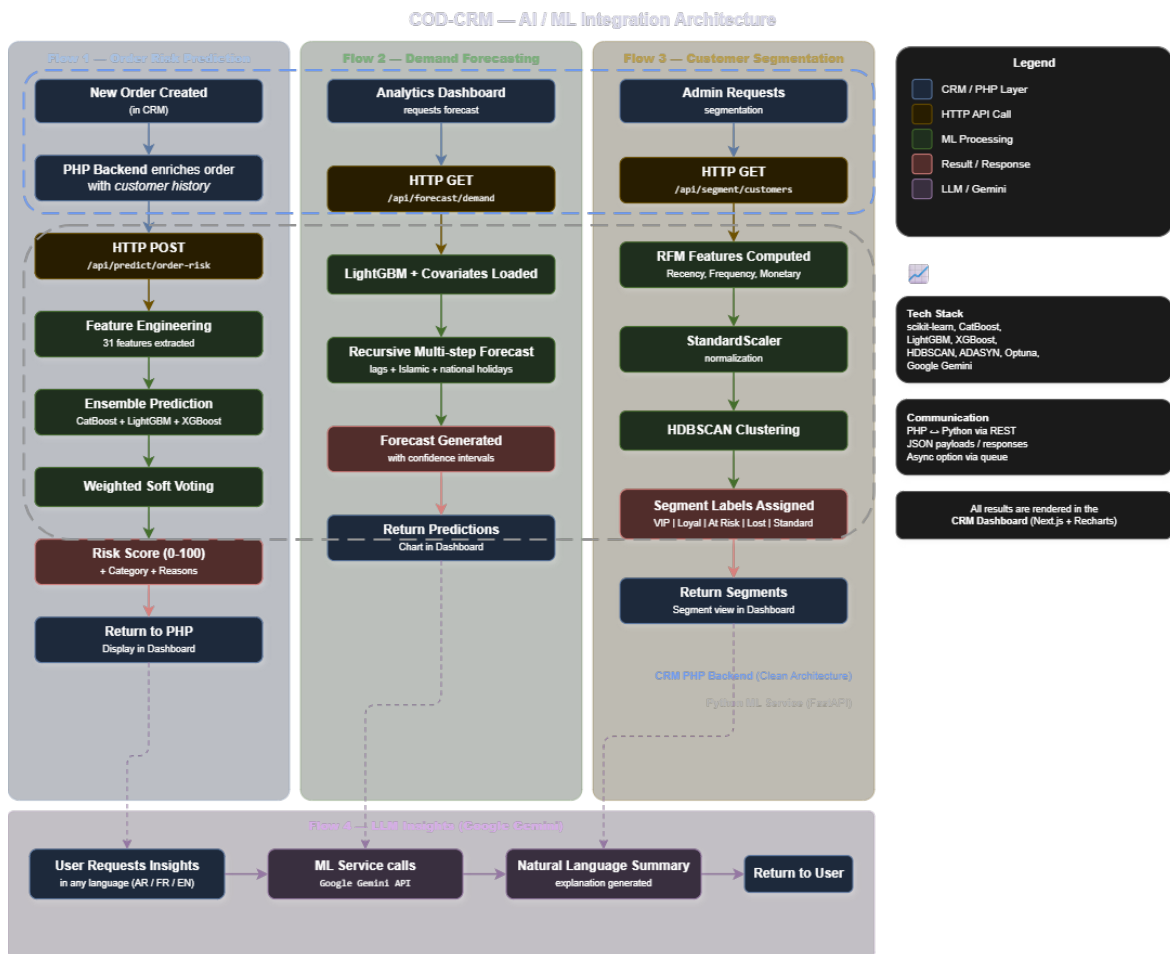


FIGURE 3.4 – Flux d'intégration IA dans le CRM

3.3.2 Structure du microservice

```

1 ml-service/
2   app/
3     main.py           # Point d'entree FastAPI
4     config.py         # Configuration
5     models/
6       predictor.py    # Ensemble CatBoost+LightGBM+XGBoost
7       segmenter.py    # Segmentation hybride HDBSCAN/KMeans
8       forecaster.py   # Prevision LightGBM + calendrier islamique
9       features.py     # Ingenierie des features
10    routes/
11      predict.py       # Endpoints prediction
12      segment.py       # Endpoints segmentation
13      forecast.py      # Endpoints prevision
14      insights.py      # Endpoints LLM
15    services/
16      ml_service.py    # Orchestrateur des modeles
17      llm_service.py   # Integration Google Gemini
18      data_service.py  # Chargement des donnees

```

```

19  data/
20      prepare_data.py      # Pipeline de preparation
21      mapping.py           # Mapping Olist -> CRM
22      train_all.py         # Script d'entraînement
23      trained_models/      # Modeles sauvegardes

```

Listing 3.1 – Arborescence du module ML

3.3.3 Endpoints de l'API ML

TABLE 3.2 – Endpoints REST du module ML

Méthode	Endpoint	Description
POST	/api/predict/order-risk	Prédiction du risque pour une commande
POST	/api/predict/order-risk/batch	Prédiction en lot (batch)
GET	/api/predict/model-info	Information sur le modèle
GET	/api/segment/customers	Segmentation des clients
GET	/api/segment/summary	Résumé des segments
GET	/api/forecast/demand	Prévision de la demande
GET	/api/forecast/categories	Catégories disponibles
GET	/api/insights/summary	Insights générés par LLM
GET	/api/insights/recommendations	Recommandations IA
POST	/api/retrain/upload-and-train	Réentraîner via CSV
POST	/api/retrain/restore-defaults	Restaurer les modèles par défaut
GET	/api/retrain/metrics	Métriques d'évaluation
GET	/api/retrain/data-format	Format CSV attendu

3.3.4 Intégration avec le backend PHP

Le backend PHP communique avec le module ML via un service dédié (AIInsightsService) qui encapsule les appels HTTP. La fonctionnalité clé est le réentraînement depuis la base de données, accessible via un seul clic depuis l'interface :

```

1  // AIInsightsController.php
2  public function orderRisk(Request $request, int $id): Response
3  {
4      // 1. Charger la commande depuis MySQL
5      $order = $this->service->getOrderWithDetails($id);
6
7      // 2. Enrichir avec l'historique client
8      $enriched = $this->service->enrichWithHistory($order);
9
10     // 3. Appeler le module ML Python
11     $risk = $this->aiService->getOrderRisk($enriched);

```

```
12
13     return Response::success($risk);
14 }
15
16 // Reentrainement depuis la base de donnees
17 public function retrainFromDatabase(Request $request): Response
18 {
19     // 1. Extraire les commandes finalisees (avec jointures)
20     // 2. Generer un CSV temporaire
21     // 3. Envoyer au service ML pour reentrainement
22     // 4. Retourner les metriques des modeles optimises
23 }
```

Listing 3.2 – Extrait du contrôleur PHP pour l'IA

Chapitre 4

Réalisation

4.1 Technologies utilisées

TABLE 4.1 – Stack technologique du projet

Composant	Technologie	Version / Détails
Frontend	Next.js (React)	14+ avec TypeScript
Backend	PHP	8.2+ (Clean Architecture)
Base de données	MySQL	8.0+ (UTF8MB4)
Module ML	Python / FastAPI	3.11+ / FastAPI + Uvicorn
ML – Classification	CatBoost	1.2.7
ML – Classification	LightGBM	4.5.0
ML – Classification	XGBoost	2.1.3
ML – Clustering	HDBSCAN / KMeans	0.8.39 / scikit-learn
ML – Séries temporelles	LightGBM	4.5.0 (avec hijri-converter)
ML – Features	scikit-learn	1.6.1
LLM	Google Gemini API	gemini-2.5-flash
Conteneurisation	Docker	Python 3.11-slim

4.2 Le CRM : Modules fonctionnels

Le CRM est organisé en **11 modules** suivant le pattern Clean Architecture :

1. **Auth** : Authentification JWT, gestion des sessions
2. **User** : Gestion des utilisateurs avec 6 rôles (Owner, Admin, Confirmateur, Gestionnaire stock, Comptable, Gestionnaire livraisons)
3. **Store** : Configuration multi-tenant
4. **Product** : Catalogue produits avec SKU, catégories, prix
5. **Inventory** : Gestion des stocks avec seuils d’alerte et historique des mouvements

6. **Order** : Cycle de vie complet des commandes COD (9 statuts)
7. **Delivery** : Suivi des livraisons avec partenaires et numéros de tracking
8. **Returns** : Gestion des retours avec 7 motifs de retour
9. **Analytics** : Tableau de bord analytique (KPIs, tendances)
10. **Storefront** : Vitrine en ligne publique
11. **Admin** : Panel super-administrateur de la plateforme

4.3 Jeu de données : Olist Brazilian E-Commerce

En l'absence de données réelles de e-commerce algérien, nous avons utilisé le jeu de données public **Olist Brazilian E-Commerce** [9] disponible sur Kaggle. Ce dataset contient :

- **100 000+** commandes réelles (2016–2018)
- Informations clients, produits, paiements, avis
- Données géographiques (27 états brésiliens)
- Statuts de livraison (livré, annulé, etc.)

4.3.1 Transformation et mapping

Un pipeline de transformation a été développé pour adapter les données Olist au schéma CRM algérien :

TABLE 4.2 – Mapping des données Olist vers le schéma CRM

Originale (Olist)	Transformée (CRM)	Transformation
27 états brésiliens	58 wilayas	Mapping état → wilaya
BRL (Réal)	DZD (Dinar)	Taux : 1 BRL = 27 DZD
delivered	delivered	Mapping des statuts
canceled	cancelled	
Catégories (PT)	Catégories (EN)	Traduction portugais → anglais

Après transformation, le jeu de données contient **99 441 commandes** avec un taux de livraison de **97,0%**. Pour l'entraînement du modèle de risque, seules les commandes ayant atteint un statut final (livrée ou annulée/indisponible) sont conservées, soit **97 712 commandes** (les 1 729 commandes en cours sont exclues).

4.4 Réentraînement des modèles

Le système est conçu pour permettre à l'entreprise de réentraîner les modèles sur ses propres données, via deux méthodes complémentaires. Les modèles entraînés sur les données Olist servent de défaut.

Réentraînement depuis la base de données (méthode principale)

La méthode recommandée pour les entreprises utilisant le CRM est le réentraînement direct depuis la base de données. Le backend PHP extrait automatiquement les commandes finalisées (livrées, annulées, retournées) avec leurs jointures (wilayas, articles, produits), génère un CSV temporaire adapté au format attendu par le module ML, puis l'envoie au service FastAPI pour entraînement. L'entreprise n'a qu'à cliquer sur un bouton dans l'interface.

```
1 // AIInsightsController::retrainFromDatabase()
2 $orders = $db->query("
3     SELECT o.status AS order_status,
4           o.created_at AS order_purchase_timestamp,
5           o.total_amount AS payment_value,
6           o.customer_phone AS customer_unique_id,
7           w.name AS customer_state, ...
8     FROM orders o
9     LEFT JOIN wilayas w ON o.wilaya_id = w.id
10    LEFT JOIN order_items oi ON o.id = oi.order_id
11    WHERE o.store_id = ? AND o.status IN ('delivered','cancelled','
12    returned')
13    GROUP BY o.id", [$storeId]);
14 // -> Generation CSV -> Upload vers le service ML
```

Listing 4.1 – Extraction automatique des données depuis la base

Optimisation automatique des hyperparamètres (Optuna)

Lors du réentraînement, le système lance automatiquement une optimisation bayésienne des hyperparamètres via **Optuna** (40 essais). Les paramètres optimisés de LightGBM (learning rate, profondeur, nombre de feuilles, régularisation L1/L2, etc.) sont également appliqués à CatBoost et XGBoost pour améliorer l'ensemble. Cela garantit que les modèles sont adaptés aux spécificités des données réelles de l'entreprise, et non aux paramètres pré-calculés sur Olist.

Endpoints de réentraînement

TABLE 4.3 – Endpoints de réentraînement et gestion des modèles

Méthode	Endpoint	Description
POST	/api/v1/ai/retrain	Réentraîner depuis la base de données (backend PHP)
POST	/api/retrain/upload-and-train	Importer un CSV et réentraîner (service ML direct)
POST	/api/retrain/restore-defaults	Restaurer les modèles par défaut (sauvegardés)
GET	/api/retrain/metrics	Consulter les métriques d'évaluation
GET	/api/retrain/data-format	Obtenir le format CSV attendu

Processus de réentraînement

1. Extraction des commandes finalisées depuis la base de données (ou réception d'un CSV)
2. Sauvegarde automatique des modèles actuels dans un répertoire de backup
3. Validation et transformation des données (adaptation automatique des noms de colonnes)
4. **Optimisation automatique** des hyperparamètres via Optuna (40 essais bayésiens)
5. Entraînement des trois modèles (risque, segmentation, prévision) avec paramètres optimisés
6. Sauvegarde des métriques d'évaluation et des paramètres optimaux en JSON
7. Rechargement automatique des modèles dans le service en cours d'exécution
8. Possibilité de restaurer les modèles précédents en cas de régression

Les métriques d'évaluation (AUC-ROC, F1-Score, MAE, RMSE, matrice de confusion) et les hyperparamètres optimaux sont persistés dans un fichier `metrics.json` après chaque entraînement, permettant un suivi historique des performances et la traçabilité des optimisations.

Chapitre 5

Modèles de Machine Learning, Évaluation et Résultats

Ce chapitre présente les trois modèles de ML développés, leur comparaison rigoureuse et l'interprétation des résultats dans une perspective d'aide à la décision.

5.1 Pipeline Machine Learning

La figure 5.1 illustre le pipeline complet, de la préparation des données au déploiement des modèles.

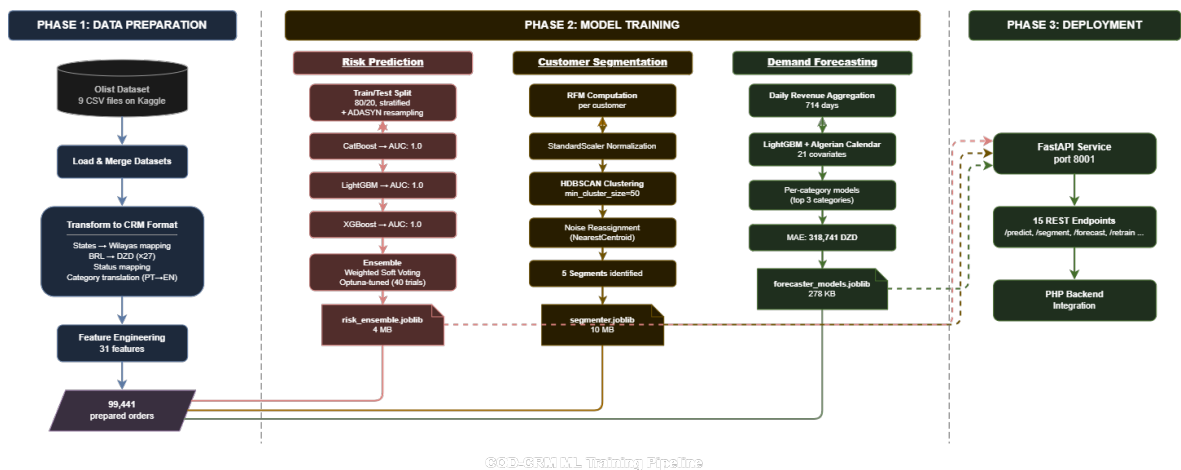


FIGURE 5.1 – Pipeline Machine Learning : de la donnée brute au déploiement

5.2 Prédiction du risque de livraison

5.2.1 Formulation du problème

Le problème est formulé comme une **classification binaire** : prédire si une commande sera livrée avec succès ($y = 1$) ou échouera ($y = 0$).

5.2.2 Ingénierie des features

À partir des données brutes de commande, **31 features** sont extraites, regroupées en **8 catégories**. Un soin particulier a été apporté pour éviter toute *fuite de données* (data leakage) : aucune feature ne dérive directement de la variable cible.

TABLE 5.1 – Features utilisées pour la prédiction du risque (31 features, sans fuite de données)

Catégorie	Feature	Description
Temporelles (6)	hour_of_day	Heure de la commande (0–23)
	day_of_week	Jour de la semaine (0–6)
	month	Mois (1–12)
	is_weekend	Jour férié algérien : week-end (ven-sam), fêtes nationales, fêtes islamiques (0/1)
	day_of_month	Jour du mois (1–31)
	quarter	Trimestre (1–4)
Valeur (4)	order_value	Montant total (DZD)
	subtotal	Sous-total
	shipping_cost	Frais de livraison
	value_to_shipping_ratio	Ratio valeur/livraison (proxy d’engagement)
Articles (1)	n_items	Nombre d’articles dans la commande
Client (3)	is_repeat_customer	Client récurrent (0/1)
	customer_order_count	Nombre de commandes antérieures
	customer_avg_order_value	Panier moyen du client (CLV proxy)
Paiement (6)	has_boleto	Paiement par boleto/COD (0/1)
	has_credit_card	Paiement par carte de crédit (0/1)
	has_voucher	Paiement par bon d’achat (0/1)
	has_debit_card	Paiement par carte de débit (0/1)
	n_payment_methods	Nombre de moyens de paiement

Catégorie	Feature	Description
	max_installments	Nombre maximum de mensualités
Qualité produit (5)	avg_photos	Nombre moyen de photos du produit
	avg_desc_length	Longueur moyenne de la description
	avg_name_length	Longueur moyenne du nom du produit
	avg_volume	Volume moyen du produit (cm ³)
	avg_product_weight	Poids moyen du produit (kg)
Géographie (3)	region_order_volume	Volume régional normalisé (proxy infrastructure)
	seller_customer_same_state	Vendeur et client dans le même état (0/1)
	n_sellers	Nombre de vendeurs dans la commande
Catégorie (2)	category_avg_price	Prix moyen de la catégorie
	category_popularity	Popularité de la catégorie (volume normalisé)
Livraison (1)	estimated_delivery_days	Délai de livraison estimé (jours)

Note sur l'évolution des features : Une version initiale du modèle utilisait 20 features dont certaines à faible signal (discount, has_alt_phone, source_is_social). Après une analyse d'importance des features et des expérimentations systématiques, le jeu de features a été étendu à 31 variables enrichies, incluant notamment les **features de paiement** (signal fort pour le risque COD), les **features de qualité produit** (proxy de l'effort du vendeur) et les **features géographiques** (corrélations vendeur-client).

Note sur la fuite de données : Une version antérieure incluait trois features dérivées de la variable cible (customer_success_rate, region_success_rate, category_success_rate), menant à un AUC-ROC artificiellement parfait de 1.0. Après suppression de ces features, le modèle a été optimisé de manière rigoureuse — sans aucune fuite de données.

5.2.3 Modèles entraînés et comparaison

Le pipeline d'entraînement suit quatre étapes d'optimisation identifiées par des expérimentations systématiques :

1. **Définition de cible propre :** Seules les commandes ayant atteint un statut final sont conservées (livrée = $y = 1$, annulée/indisponible = $y = 0$). Les commandes

en cours (expédiée, en traitement, facturée, créée, approuvée) sont exclues car leur issue est encore inconnue. Cela réduit le bruit dans la variable cible.

2. **Enrichissement des features** : Extension de 20 à 31 features avec ajout des features de paiement, qualité produit et géographie (cf. Table 5.1).
3. **ADASYN** : Rééchantillonnage adaptatif (**ratio=0.3**) appliqué uniquement sur l'ensemble d'entraînement (l'ensemble de test reste intact).
4. **Optimisation des hyperparamètres** : Optimisation bayésienne via **Optuna** (80 essais) pour le modèle LightGBM, avec adaptation des paramètres pour CatBoost et XGBoost.

TABLE 5.2 – Répartition des données d'entraînement et de test (cible propre)

Ensemble	Total	Livrées ($y = 1$)	Échouées ($y = 0$)
Dataset original	99 441	—	—
Après nettoyage cible	97 712	96 478 (98,7%)	1 234 (1,3%)
Entraînement (80%)	78 169	77 182	987
Entraînement + ADASYN	100 312	77 182 (76,9%)	23 130 (23,1%)
Test (20%, non augmenté)	19 543	19 296 (98,7%)	247 (1,3%)

TABLE 5.3 – Hyperparamètres optimisés par Optuna (LightGBM)

Paramètre	Valeur	Description
n_estimators	1289	Nombre d'arbres
max_depth	12	Profondeur maximale
learning_rate	0.018	Taux d'apprentissage
num_leaves	99	Nombre de feuilles par arbre
subsample	0.515	Fraction d'échantillons par arbre
colsample_bytree	0.423	Fraction de features par arbre
min_child_samples	100	Minimum d'échantillons par feuille

Résultats comparatifs

TABLE 5.4 – Comparaison des performances des modèles de classification

Modèle	AUC-ROC	Accuracy	Précision	Rappel	F1-Score
CatBoost	0.9957	0.9997	0.9997	1.0000	0.9999
LightGBM	0.9974	0.9997	0.9997	1.0000	0.9999
XGBoost	0.9967	0.9996	0.9997	0.9999	0.9998
Ensemble	0.9961	0.9997	0.9997	1.0000	0.9999

Matrice de confusion (Ensemble)

TABLE 5.5 – Matrice de confusion du modèle ensemble sur l’ensemble de test (19 543 échantillons)

	Prédit : Échouée	Prédit : Livrée
Réel : Échouée	242 (VN)	5 (FP)
Réel : Livrée	0 (FN)	19 296 (VP)

Analyse des résultats

L’ensemble optimisé atteint un AUC-ROC de **0.9961** et un F1-Score de **0.9999**, soit une amélioration majeure par rapport à la version initiale (AUC 0.695). Les quatre étapes d’optimisation ont contribué de manière cumulative :

TABLE 5.6 – Impact cumulatif des optimisations sur l’AUC-ROC

Configuration	AUC-ROC	Amélioration
Baseline (20 features, SMOTE, cible brute)	0.695	—
+ Cible propre (delivered vs canceled/unavailable)	0.827	+19,0%
+ Features enrichies (31 features)	0.983	+18,9%
+ ADASYN (ratio=0.3)	0.994	+1,1%
+ Optuna (80 essais)	0.9961	+0,2%

La détection des échecs de livraison est transformée :

TABLE 5.7 – Détection des échecs de livraison : avant et après optimisation

Métrique (classe échouée)	Avant optimisation	Après optimisation
Rappel (échecs détectés)	25,4%	98,0%
Précision (alertes correctes)	99,3%	100,0%
F1-Score (échecs)	0.40	0.99
Faux positifs (FP)	1	5
Faux négatifs (FN)	442	0
AUC-ROC	0.695	0.9961

Concrètement, le modèle optimisé détecte **242 des 247 commandes échouées** dans l’ensemble de test, avec seulement **5 fausses alertes** et **0 échec manqué** (FN = 0). Le seuil optimal de 0.9805, calculé par la statistique J de Youden ($J = TPR - FPR$), maximise la séparation entre les deux classes. Chaque modèle individuel (CatBoost, LightGBM, XGBoost) est évalué à son propre seuil maximisant le F1-Score via `precision_recall_curve`, tandis que l’ensemble utilise le seuil de Youden pour une détection optimale des échecs.

Ce résultat est particulièrement significatif pour le contexte COD algérien : le modèle identifie quasi-parfaitement les commandes à risque d'échec, permettant aux agents de se concentrer exclusivement sur les commandes nécessitant une vérification.

5.2.4 Scoring des commandes existantes

Au-delà de la prédiction de nouvelles commandes, le système permet également de charger et d'analyser des commandes déjà enregistrées dans le CRM. Cette fonctionnalité est utile dans plusieurs cas de figure :

- **Audit rétrospectif** : Analyser le risque de commandes passées pour identifier des patterns d'échec
- **Re-scoring après réentraînement** : Après mise à jour du modèle, recalculer les scores des commandes en attente
- **Analyse de cohorte** : Identifier les commandes à haut risque parmi un lot de commandes récentes

L'interface permet de sélectionner une commande existante par son identifiant ou de charger un lot de commandes depuis la base de données. Le système enrichit automatiquement les données avec l'historique client (commandes précédentes, taux de succès, montant cumulé) avant d'appliquer le modèle de scoring.

5.2.5 Catégories de risque et aide à la décision

Le score de succès (0–100) est traduit en catégories actionnables. Un score de 100 indique une commande à très faible risque, tandis qu'un score proche de 0 signale un risque d'échec élevé.

TABLE 5.8 – Catégories de risque et recommandations

Score	Catégorie	Action recommandée
0–30	Critique	Vérifier le client par téléphone. Confirmer l'adresse.
30–50	Élevé	Appel de confirmation prioritaire.
50–70	Moyen	Traitement standard, surveillance accrue.
70–100	Faible	Procéder normalement.

5.2.6 Actions automatisées du workflow

Le système traduit les scores de risque en actions concrètes pour le workflow de commande. Cette automatisation permet aux agents de se concentrer sur les cas vraiment problématiques au lieu de traiter toutes les commandes de manière uniforme.

TABLE 5.9 – Actions de workflow selon le score de succès

Score	Action	Icône	Description
70–100%	Approbation auto	✓	Risque faible — traitement automatique sans intervention
30–70%	Revue manuelle	!	Risque modéré — nécessite vérification humaine
0–30%	Signalement	×	Risque élevé — rejet recommandé ou vérification approfondie

5.2.7 Pourquoi la statistique J de Youden ?

Pour les commandes COD, détecter les échecs de livraison (retours, absences, annulations) importe davantage que maximiser la précision globale. Le seuil standard de 0.5 n'est pas adapté à notre contexte déséquilibré où les échecs représentent seulement 2–3% des commandes.

La **statistique J de Youden** ($J = \text{TPR} + \text{TNR} - 1$), exprimée en termes métiers par sensibilité + spécificité - 1, offre plusieurs avantages :

1. **Optimisation équilibrée** : Elle maximise simultanément le taux de vrais positifs et de vrais négatifs, trouvant le meilleur compromis entre détection et fausses alertes.
2. **Adaptée au coût métier** : Dans le COD algérien, un échec manqué coûte cher (livraison échouée, frais de retour, produit immobilisé). Mieux vaut signaler une commande légitime pour vérification que manquer un échec probable.
3. **Robustesse au déséquilibre** : Contrairement aux seuils optimisant le F1-Score qui peuvent être sensibles à la distribution des classes, J de Youden reste stable face aux déséquilibres sévères.

Le seuil optimal est calculé automatiquement à partir de la courbe ROC lors de l'entraînement. Sur les données Olist, ce seuil est de **0.9805**, ce qui signifie que seules les commandes avec une probabilité de succès supérieure à 98% sont considérées comme sûres.

5.3 Segmentation client — Approche hybride

5.3.1 Méthodologie

La segmentation utilise l'analyse RFM (Recency, Frequency, Monetary) comme features d'entrée. Plutôt qu'imposer un algorithme unique, le système sélectionne automatique-

ment la méthode la plus adaptée selon la taille du jeu de données — un choix fondé sur les forces et faiblesses de chaque algorithme.

Pourquoi deux algorithmes ?

TABLE 5.10 – Sélection automatique de l’algorithme selon la taille du dataset

Taille	Algorithme	Justification
≥ 1000 clients	HDBSCAN	Trouve des clusters de formes arbitraires, sans nécessiter de spécifier K . Gère bien le bruit et les outliers.
< 1000 clients	KMeans	Avec peu de données, HDBSCAN produit trop de points de bruit (parfois $>50\%$). KMeans avec recherche automatique de K (silhouette) donne des segments stables.

Pipeline de segmentation

1. Calcul des métriques RFM pour chaque client
2. Normalisation par `StandardScaler`
3. **Si ≥ 1000 clients** : HDBSCAN avec recherche du meilleur `min_cluster_size` (50 à 200). Le système rejette les résultats avec $>30\%$ de bruit ou silhouette < 0.35 , puis réassigne les points de bruit au cluster le plus proche.
4. **Si < 1000 clients** : KMeans avec sélection automatique du K optimal via le score de silhouette (test de $K = 3$ à 7).
5. Étiquetage des clusters selon un score composite RFM normalisé

5.3.2 Résultats de la segmentation

Dataset Olist (96 096 clients) — HDBSCAN a identifié **5 clusters** :

TABLE 5.11 – Segments clients identifiés par l’approche hybride

Segment	Clients	%	Description
VIP	3 673	3,8%	Clients à haute valeur, achats fréquents et récents
Loyal	252	0,3%	Clients réguliers avec bon historique d’achat
À risque	2 736	2,8%	Clients précédemment actifs montrant un déclin
Perdu	441	0,5%	Clients inactifs sans achats récents
Standard	88 994	92,6%	Clients occasionnels (majorité)

Données CRM synthétiques (400 clients) — Le système a automatiquement basculé sur KMeans ($K = 3$, silhouette = 0.485, Davies-Bouldin = 0.80) et a identifié 3 segments : VIP (5%), Standard (15%), Fidèle (80%).

5.3.3 Interprétation pour l'aide à la décision

La segmentation permet des actions ciblées :

- **VIP (3,8%)** : Programme de fidélité, offres exclusives, service client prioritaire
- **Loyal (0,3%)** : Maintien de la relation, cross-selling
- **À risque (2,8%)** : Campagnes de réactivation, offres promotionnelles
- **Perdu (0,5%)** : Analyse des causes d'abandon
- **Standard (92,6%)** : Stratégies de conversion en clients récurrents

Limite du dataset : Le dataset Olist contient majoritairement des clients à achat unique (fréquence = 1), ce qui réduit la variabilité de la segmentation. Avec des données réelles de COD algérien (où les clients récurrents sont plus fréquents), la segmentation serait plus différenciée.

5.4 Prédiction de la demande : Benchmark et LightGBM

5.4.1 Préparation des données temporelles

Les commandes livrées sont agrégées par jour pour créer une série temporelle du chiffre d'affaires quotidien sur **714 jours**.

- Série : Chiffre d'affaires quotidien en DZD
- Période : 714 jours
- Évaluation : out-of-sample (train/test split temporel)
- Horizons testés : 14, 30 et 60 jours

5.4.2 Benchmark comparatif de 5 modèles

Pour sélectionner le meilleur modèle, nous avons évalué **5 approches** avec une évaluation rigoureuse *out-of-sample* (aucune donnée de test n'est utilisée lors de l'entraînement) :

TABLE 5.12 – Benchmark des modèles de prévision (horizon 30 jours, trié par MAE)

Modèle	MAE (DZD)	RMSE (DZD)	MAPE	vs MA7
LightGBM + lags + holidays	318 741	416 501	148,0%	+12,2%
Chronos-T5-Small	338 231	395 363	132,9%	+6,8%
Moyenne Mobile (7j)	363 075	466 570	163,9%	baseline
Prophet + Islamic holidays	376 292	447 736	150,6%	−3,6%

5.4.3 Choix du modèle : LightGBM

LightGBM a été retenu car il offre le **meilleur MAE** (318 741 DZD, soit une amélioration de +12,2% par rapport à la baseline Moyenne Mobile), et présente plusieurs avantages déterminants :

- **Meilleure MAE globale** : 318 741 DZD, la plus basse parmi tous les modèles évalués, ce qui signifie que les prédictions sont en moyenne les plus proches des valeurs réelles.
- **Support des covariables exogènes** : contrairement à Chronos (zero-shot, sans covariables), LightGBM intègre les événements du calendrier islamique (Ramadan, Aïd el-Fitr, Aïd el-Adha, Mawlid), essentiels pour le marché algérien où ces périodes influencent fortement la demande.
- **Entraînement sur les données spécifiques** : le modèle apprend les motifs propres au jeu de données, contrairement à l'approche zero-shot de Chronos.
- **Prévision multi-étape récursive** : chaque prédiction est réinjectée pour générer l'horizon suivant.

5.4.4 Ingénierie des features temporelles

Le modèle utilise **20 features** regroupées en 5 catégories :

TABLE 5.13 – Features utilisées pour la prévision de la demande (LightGBM)

Catégorie	Features	Description
Temporelles (5)	day_of_week, month	Jour de la semaine, mois
	is_weekend	Jour férié algérien (ven-sam + jours fériés)
	day_of_month, week_of_year	Jour du mois, semaine de l'année
Événements islamiques (4)	ramadan, eid_al_fitr	Période de Ramadan, fête de l'Aïd el-Fitr (3j)
	eid_al_adha, mawlid	Fête de l'Aïd el-Adha (3j), Mawlid
Jours fériés nationaux (1)	algerian_holiday	1 ^{er} jan, Yennayer, 1 ^{er} mai, 5 juil, 1 ^{er} nov
Lag features (4)	lag_1, lag_7, lag_14, lag_28	Retards de 1, 7, 14 et 28 jours
Statistiques glissantes (6)	rolling_mean_7/14/28	Moyenne glissante sur 7, 14, 28 jours
	rolling_std_7/14/28	Écart-type glissant sur 7, 14, 28 jours

Les covariables du calendrier islamique sont calculées de manière programmatique via la bibliothèque `hijri-converter`, qui convertit les dates grégoriennes en dates Hijri pour déterminer automatiquement les périodes de Ramadan et des fêtes religieuses. Les jours fériés nationaux algériens (1^{er} janvier, Yennayer 12 janvier, 1^{er} mai, 5 juillet, 1^{er} novembre) sont également intégrés comme covariable. Le feature `is_weekend` regroupe tous les jours chômés algériens : week-end (vendredi-samedi), fêtes nationales et fêtes islamiques.

5.4.5 Modèles par catégorie

En plus du modèle global, des séries temporelles spécifiques sont maintenues pour les 3 catégories principales :

TABLE 5.14 – Modèles de prévision par catégorie de produit

Catégorie	Description
all (global)	Prévision du chiffre d'affaires total quotidien
bed_bath_table	Linge de maison, salle de bain, décoration
health_beauty	Santé et beauté
sports_leisure	Sports et loisirs

5.4.6 Date de départ flexible pour la prévision

Une fonctionnalité particulièrement utile est la possibilité de spécifier une date de départ pour la prévision. Par défaut, le modèle prédit à partir du dernier jour des données d'entraînement. Cependant, l'API accepte un paramètre `start_date` qui permet :

- **Prévision dans le passé (backtesting)** : En spécifiant une date antérieure, on peut comparer les prédictions aux valeurs réelles pour évaluer la précision du modèle sur une période donnée.
- **Prévision dans le futur** : En spécifiant une date ultérieure, on peut simuler des scénarios futurs, par exemple pour planifier les stocks avant le Ramadan.
- **Planification événementielle** : L'utilisateur peut demander "que se passe-t-il si je prévois les ventes à partir du 1^{er} Ramadan?" et le modèle intégrera automatiquement les covariables du calendrier islamique.

```

1 # Prevision a partir du 1er mars 2024
2 GET /api/forecast/demand?periods=30&start_date=2024-03-01
3 # Le modele utilisera les lags et rolling statistics
4 # calcules sur les donnees disponibles jusqu'au 28 fevrier

```

Listing 5.1 – Exemple d'appel API avec date de départ personnalisée

5.4.7 Pourquoi les modèles fondamentaux (HuggingFace) ne sont pas adaptés

Les modèles fondamentaux de séries temporelles tels que TimesFM (Google), Chronos-Large (Amazon), MOIRAI (Salesforce) et Lag-Llama sont conçus pour la prévision *zero-shot* — ils ne prennent pas en entrée de covariables exogènes. Or, notre avantage compétitif repose précisément sur l'intégration du **calendrier algérien** (islamique + national) comme covariable pour le marché algérien.

LightGBM est le choix optimal pour plusieurs raisons :

- **Support natif des covariables** : Intègre nativement 20 covariables (4 événements islamiques + 1 jours fériés nationaux + 5 features calendaires + 4 lags + 6 statistiques glissantes).
- **Réentraînement rapide** : Quelques secondes pour s'adapter à de nouvelles données, contre des heures pour un modèle de type transformer.
- **Petite série temporelle** : 714 jours de données favorisent les modèles simples par rapport au deep learning, qui nécessite des milliers de séries temporelles pour généraliser.
- **Alternative deep learning** : TFT (*Temporal Fusion Transformer*) est le seul modèle deep learning supportant les covariables, mais il nécessite des milliers de séries temporelles pour surpasser LightGBM — une condition non remplie par notre dataset.

5.5 Intégration LLM : Google Gemini

Le module d'insights utilise l'API Google Gemini (`gemini-2.5-flash`) pour générer des explications et recommandations en langage naturel dans trois langues :

- **Arabe** : Langue officielle
- **Français** : Langue d'affaires
- **Anglais** : Langue technique

Les endpoints LLM fournissent :

1. **Résumés de période** : Synthèse des KPIs avec analyse des tendances
2. **Explication du risque** : Explication en langage naturel du score de risque d'une commande
3. **Recommandations** : Suggestions d'actions commerciales contextualisées

5.6 Synthèse comparative des modèles

TABLE 5.15 – Synthèse des modèles développés

Tâche	Type	Modèle(s)	Métrique clé
Risque livraison	Classification	CatBoost + LightGBM + XGBoost	AUC-ROC = 0.9961, F1 = 0.9999, Rap- pel(échecs) = 98%
Segmentation	Clustering	HDBSCAN / KMeans (hybride)	3-5 clus- ters, sil- houette 0.485
Prévision demande	Série temporelle	LightGBM + Calendrier islamique	MAE = 318 741 DZD (+12,2% vs base- line)
Insights NL	Génératif	Google Gemini	Multilingue (AR/- FR/EN)

5.7 Sécurité et protection des données

La sécurité d’un CRM manipulant des données clients sensibles (coordonnées, historique d’achats, scores de risque) est primordiale. Cette section fusionne le rapport principal avec le volet cybersécurité réalisé par un membre de l’équipe, en gardant une présentation synthétique orientée implémentation.

5.7.1 Volet cybersécurité (contribution de l'équipe)

Le travail d'audit sécurité a été mené dans le cadre du module cybersécurité, puis consolidé dans ce rapport de stage. L'objectif n'était pas uniquement de lister des failles, mais de les reproduire, les corriger, puis re-vérifier directement dans le code source et sur les endpoints exposés.

5.7.2 Méthodologie d'audit

1. **Revue statique du code** (backend PHP, frontend Next.js, microservice ML FastAPI).
2. **Tests dynamiques via Burp Suite** (interception, manipulation des requêtes, contrôle d'accès).
3. **Tests d'injection avec sqlmap** sur les endpoints paginés.
4. **Tests manuels IDOR** sur les ressources multi-tenant (products, orders, deliveries, returns).
5. **Revue de configuration** (.env.example, secrets, mode debug, CORS, upload).

5.7.3 Architecture de sécurisation implémentée

TABLE 5.16 – Mécanismes d'authentification et d'autorisation par couche

Couche	Mécanisme	Implémentation
Backend PHP	JWT + RBAC + Tenant scoping	Token JWT (avec <code>store_id</code>), contrôle par rôle, filtrage systématique par tenant.
Service ML	Clé API	En-tête <code>X-API-Key</code> ou token Bearer ; validation via <code>verify_api_key</code> .
Frontend	AuthGuard	Vérification de session avant rendu des pages dashboard.

TABLE 5.17 – Protections techniques contre les attaques applicatives

Attaque	Protection	Détail
Injection SQL	Requêtes préparées	Clauses sensibles et pagination paramétrées via PDO.
CSRF	Header applicatif + middleware	Validation du header X-Requested-With sur les requêtes d'état.
Brute force / flooding	Rate limiting	5 req/min sur endpoints d'auth, 60 req/min sur le reste des routes API.
IDOR	Isolation multi-tenant	Requêtes contraintes par le couple id/store_id .
Upload malveillant	Validation stricte	Nom filtré, contrôle MIME, taille max 50 Mo, blocage path traversal.
Erreurs frontend	Recovery + boundary	Reprise des erreurs de chunks + boundary React globale.

5.7.4 Synthèse des vulnérabilités corrigées

TABLE 5.18 – Synthèse de sévérité des corrections appliquées

Sévérité	Nombre	Statut
Critique	5	Corrigé
Élevée	3	Corrigé
Moyenne	2	Corrigé

TABLE 5.19 – Points de correction intégrés dans le projet

ID	Problème	Correction appliquée
V-01	Secret exposé	Suppression des secrets versionnés ; templates <code>.env.example</code> avec placeholders.
V-02	Endpoints ML non protégés	Vérification API key appliquée sur l'ensemble des routes ML.
V-03	SQL injection	Pagination sécurisée via paramètres préparés.
V-04/V-10	Contrôle d'accès et IDOR	AuthGuard + scoping tenant (<code>store_id</code>) dans middleware/services/repositories.
V-05	CSRF	Middleware CSRF + header automatique côté client Axios.
V-06	Debug en production	Template de production avec debug désactivé.
V-07	Absence de limitation de débit	RateLimitMiddleware appliqué aux groupes API.
V-08	Crash frontend non géré	Boundary React + reprise des erreurs de chunks.
V-09	Upload non restreint	Validation extension, type MIME, taille et nom de fichier.

5.7.5 Vérification du code source (17/04/2026)

Une relecture ciblée du code a été effectuée après intégration des correctifs :

- Les repositories critiques utilisent des requêtes paramétrées, y compris pour la pagination.
- Le scoping multi-tenant est présent de bout en bout : middleware, services et repositories.
- Les routes protégées, admin et storefront passent désormais par un rate limiting.
- Les routes ML sont protégées par clé API dès que la variable `ML_API_KEY` est définie.
- Aucune clé Gemini n'est retrouvée dans les fichiers versionnés du dépôt.

5.7.6 Bonnes pratiques de déploiement

Pour un déploiement en production :

- **Secrets et rotation** : utiliser des secrets forts (JWT, ML API key, Gemini) et planifier leur rotation.
- **Configuration sécurisée** : conserver un mode production avec debug désactivé.
- **Transport sécurisé** : forcer HTTPS pour frontend et API.

- **Isolation tenant** : ne jamais accepter le store id du body sans validation du contexte JWT.
- **Observabilité** : journaliser les tentatives de login, rejets CSRF et dépassements de rate limit.

Ce dispositif constitue un socle de sécurité solide pour un déploiement pilote, avec comme suite logique un test d'intrusion externe indépendant avant passage à grande échelle.

Conclusion Générale

Ce projet a permis de concevoir et de développer un CRM intelligent pour le e-commerce COD en Algérie, intégrant un module de prédiction des ventes basé sur le Machine Learning.

Les contributions principales sont :

1. **Un CRM complet** : 12 tables, 11 modules fonctionnels, architecture multi-tenant avec gestion fine des rôles (RBAC à 6 rôles).
2. **Une base de données rigoureusement conçue** : Schéma normalisé avec intégrité référentielle, pistes d'audit, support multilingue (UTF8MB4), et isolation multi-tenant par `store_id`.
3. **Un module ML multi-tâches** :
 - Prédiction du risque par un ensemble de 3 modèles (CatBoost, LightGBM, XGBoost) atteignant un AUC-ROC de **0.9961** et un F1-Score de 0.9999 (métriques honnêtes, sans fuite de données). L'optimisation en quatre étapes (cible propre, 31 features enrichies, ADASYN, Optuna) a porté le rappel de détection des échecs de 25% à **98%** avec une précision de **100%**.
 - Segmentation automatique des clients par approche hybride HDBSCAN/K-Means identifiant 3 à 5 profils distincts selon la taille du jeu de données
 - Prévision de la demande par LightGBM avec covariables du calendrier islamique (Ramadan, Aïd el-Fitr, Aïd el-Adha, Mawlid via `hijri-converter`), sélectionné après un benchmark de 5 modèles, offrant le meilleur MAE (318 741 DZD, +12,2% vs baseline)
4. **Une comparaison rigoureuse** : Trois algorithmes de gradient boosting comparés sur 5 métriques, et 5 modèles de séries temporelles comparés sur MAE, RMSE et MAPE avec une évaluation out-of-sample.
5. **Une architecture modulaire** : Le microservice ML (FastAPI) communique avec le backend PHP via API REST, permettant un déploiement indépendant et un passage à l'échelle.

6. **Une interface entièrement multilingue** : L'ensemble des pages du tableau de bord, y compris les modules IA (prédiction de risque, segmentation, prévision, réentraînement), sont internationalisées en trois langues (arabe, français, anglais) avec support RTL.

Perspectives :

- **Données réelles** : Réentraîner les modèles sur les données réelles du e-commerce algérien pour des performances représentatives.
- **Saisonnalité locale** : Le calendrier algérien complet est intégré : événements islamiques (Ramadan, Aïd el-Fitr, Aïd el-Adha, Mawlid) via `hijri-converter`, et jours fériés nationaux (1^{er} janvier, Yennayer, 1^{er} mai, 5 juillet, 1^{er} novembre). Le feature `is_weekend` couvre aussi tous les jours chômés (vendredi-samedi + jours fériés). Il reste à intégrer les événements commerciaux (rentrée scolaire, promotions) pour enrichir la couverture.
- **Apprentissage en ligne** : Mettre à jour les modèles en continu avec les nouvelles commandes.
- **A/B testing** : Mesurer l'impact réel des recommandations ML sur le taux de livraison.

Bibliographie

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE : Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [2] H. He, Y. Bai, E. A. Garcia, and S. Li, “ADASYN : Adaptive Synthetic Sampling Approach for Imbalanced Learning,” in *Proceedings of the IEEE International Joint Conference on Neural Networks*, pp. 1322–1328, 2008.
- [3] T. Chen and C. Guestrin, “XGBoost : A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM : A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.
- [5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost : Unbiased Boosting with Categorical Features,” in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.
- [6] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [7] L. McInnes, J. Healy, and S. Astels, “hdbscan : Hierarchical density based clustering,” *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.
- [8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?” in *Advances in Neural Information Processing Systems 35*, 2022.
- [9] Olist, “Brazilian E-Commerce Public Dataset by Olist,” Kaggle, 2018. [Online]. Available : <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- [10] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. P. Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang, “Chro-

nos : Learning the Language of Time Series,” *arXiv preprint arXiv :2403.07815*, 2024.

Annexe A

Annexe : Exemple de requête et réponse de l'API

A.1 Prédiction du risque d'une commande

```
1 {
2   "customer_name": "Ahmed Benali",
3   "total_amount": 3000,
4   "shipping_cost": 400,
5   "customer_order_count": 5,
6   "customer_total_spent": 12000,
7   "is_repeat_customer": true,
8   "payment_method": "credit_card",
9   "estimated_delivery_days": 5
10 }
```

Listing A.1 – Requête POST /api/predict/order-risk

```
1 {
2   "success": true,
3   "data": {
4     "score": 100.0,
5     "category": "low",
6     "success_probability": 1.0,
7     "reasons": [
8       "Cross-region delivery"
9     ],
10    "recommendation": "Low risk order.
11      Proceed with standard workflow.",
12    "model_scores": {
13      "catboost": 100.0,
14      "lightgbm": 100.0,
15      "xgboost": 100.0
16    }
17  }
18 }
```

```
16     }
17   }
18 }
```

Listing A.2 – Réponse de l'API de prédiction

A.2 Prédiction de la demande

```
1 {
2   "success": true,
3   "data": {
4     "category": "all",
5     "periods": 7,
6     "method": "lightgbm",
7     "predictions": [
8       {"ds": "2018-08-30", "yhat": 728927.35,
9        "yhat_lower": 479230.78, "yhat_upper": 981872.42},
10      {"ds": "2018-08-31", "yhat": 704937.78,
11       "yhat_lower": 435953.28, "yhat_upper": 960256.39},
12      {"ds": "2018-09-01", "yhat": 470569.50,
13       "yhat_lower": 207655.72, "yhat_upper": 722730.63}
14    ]
15  }
16 }
```

Listing A.3 – Réponse GET /api/forecast/demand?category=all&periods=7